TatvaSoft

SOFTWARE DEVELOPMENT · GUIDELINE

TVS-DEV-AI-GL-01.5

AI Model Selection

Development • Testing • Code Review | Company / Client / Project standard

VERSION	EFFECTIVE DATE	PREPARED BY	APPROVED BY
1.0	28.09.2026	Badal Patel	Shital Patel

HOW TO READ THIS DOCUMENT

Short on time? Read the **Core rule** and the **Provider → model guidance** table.

Tables show task tiers, recommended models per provider, and which model and agent to use at each stage.

Parent guideline: **TVS-DEV-AI-GL-01 AI Development Tools Guidelines** (AI Model Selection section).

Document Control

Document Control Information

Document Title	AI Model Selection
Document ID	TVS-DEV-AI-GL-01.5
Document Version	Version 1.0
Effective Date	28.09.2026
Parent Document	TVS-DEV-AI-GL-01 — AI Development Tools Guidelines

Document Publication History

Document Prepared By	Badal Patel
Document Reviewed By	Jigar Prajapati
Document Approved By	Shital Patel
Initial Release Version	1.0
Initial Release Date	28.09.2026

Document Revision History

Version No.	Revision Description	Change Author	Change Approver	Revision Date
1.0	Final Release	Badal Patel	Shital Patel	25.09.2026

Document Definition

This document sets how to choose an AI model for development, testing and code review: select the lowest model that handles the task well and step up only for complexity or risk. It covers task tiers, recommended models for OpenAI, Claude, Gemini and GitHub Copilot, same vs. separate agents for testing and code review, and cost and safety rules. Use it when starting a task, setting up a project’s AI tools, or deciding if a stronger model is worth the cost.

Contents

Introduction	What this standard covers and when to use it
1. Core rule	Use the lowest model tier that reliably manages the task
2. Task tiers	Simple · Normal · Complex / Critical
3. Provider → model guidance	Claude · GitHub Copilot · OpenAI · Gemini
4. Development vs. testing vs. code review	Model, agent and developer action per stage
5. Same or separate agents for testing and code review	When to use a separate agent
6. Copy/paste prompt examples	Ready-to-use prompts by task type
7. Cost and safety rules	Explicit selection, cost control and data safety

Introduction

> **KEY TAKEAWAY**
>
> This document sets how to choose an AI model for development, testing and code review: **select the lowest model that handles the task well and step up only for complexity or risk**.

It covers **task tiers**, **recommended models** for OpenAI, Claude, Gemini and GitHub Copilot, **same vs. separate agents** for testing and code review, and **cost and safety rules**.

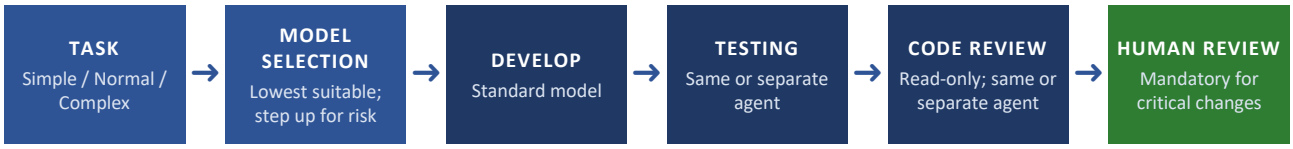
Use it when:

Starting a task	**Setting up a project's AI tools**	**Deciding if a stronger model is worth the cost**

1. Core rule

> **CORE RULE**
>
> **Use the lowest model tier that reliably manages the task.** Select the model explicitly; do not rely on Auto, which can switch to a stronger, more expensive model without your consent.
>
> Move to a stronger model only when **complexity, uncertainty, security, architecture, or business impact** justifies it. Do not use the strongest model for every task.

Workflow briefly. Findings from testing and code review go back to development. **Human approval is required** before critical changes are merged or deployed.



2. Task tiers

Tier	What it looks like	Examples
Simple	Well-defined change in one or a few files that follows an existing pattern	Add a field validation, fix a small bug, update a unit test
Normal	Feature work inside the existing architecture, spanning several files or layers	New API endpoint, CSV import using the existing service pattern
Complex / Critical	Architecture or cross-cutting design, or anything touching **security, authentication/authorization, production data, infrastructure or performance**	Asynchronous bulk import, auth changes, data migrations

3. Provider → model guidance

Provider / Tool	Simple	Normal development	Testing / Code review	Complex / Critical
Claude / Claude Code	Haiku 4.5	Sonnet 5	Sonnet 5; Opus 5.5 for high-risk changes	Opus 5.5; Fable 5.1 for the hardest reasoning work
GitHub Copilot	GPT-6 Luna or Haiku 4.5	GPT-6 Sol or Sonnet 5	Same as normal; Opus 5.5 for high-risk changes	Explicitly select Opus 5.5 or GPT-6 Astra
OpenAI / Codex	GPT-6 Luna	GPT-6 Sol	GPT-6 Sol	GPT-6 Astra
Gemini	Gemini 3.5 Flash-Lite	Gemini 3.8 Flash	Gemini 3.8 Flash	Gemini 3.1 Pro (preview[1])

[1] Gemini 3.1 Pro is still a preview model. Use it only where the Company/Client has approved preview models.

> **AVAILABILITY**
> Which models you can use depends on the **Company/Client license**, the **product surface** and **current provider support**. Model names change, so project boilerplate should use **configurable model aliases** rather than hard-coded model IDs.

4. Development vs. testing vs. code review

Stage	Recommended model	Agent	Developer action
Development	Standard tier (e.g., Sonnet 5, GPT-6 Sol, Gemini 3.8 Flash)	Development agent	Provide the requirement; approve the plan for normal and complex work
Testing	Same model as development; one level stronger is recommended	Test agent should be a separate as applicable	Run the tests; verify edge cases and coverage
Code review	Same or stronger model; stronger model and fresh context for high-risk changes	Read-only code review agent: should be a separate agent as applicable	Review the code review findings; a human decides which fixes to apply
Security / Critical	Highest approved model	Security Review Agent	**Human approval is mandatory before merge or deploy**

5. Same or separate agents for testing and code review

Testing and code review can use the same agent and model as development, or a separate agent and model, as applicable.

NORMAL AND COMPLEX CHANGES

For normal and complex changes, prefer **a separate agent with a fresh context**, because an agent checking its own work tends to confirm it rather than question it.

SIMPLE CHANGES

For very simple straightforward changes, the same development agent may work for quick test and review.

- **Testing** checks behaviour, negative cases, edge cases, and coverage.
- **Code review** checks correctness, architecture, security, performance, and maintainability.
- **Critical changes** get a stronger model and mandatory human approval.

6. Copy/paste prompt examples

Type	Prompt
Simple	Add a required-email validation to the existing Customer DTO and update the related unit test. Follow the existing project pattern.
Normal	Add CSV customer import using the existing API/service pattern. First inspect the existing Customer and import code, and create a plan. Do not modify code until the plan is clear.
Complex	Design and implement asynchronous import for up to 100,000 records, including PostgreSQL transaction strategy, retry handling, progress status, failure reporting, and Angular UI. Analyze the architecture first and create a plan.
Testing	Function as an independent test agent. Review the requirement and implementation, identify positive, negative and edge cases, add or update tests, run them, and report failures and missing coverage.
Code review	Review only the current changes. Do not modify files. Check correctness, security, performance, project conventions, and test coverage. Return findings grouped as Critical / Warning / Suggestion.

7. Cost and safety rules

- Select the lowest suitable model **explicitly**. Avoid Auto; it can move to a higher-cost model without your consent.
- Reserve stronger models for complexity or risk; **do not** move every task to the highest tier.
- Use **scripts or automation** for repetitive, deterministic work instead of asking an AI model repeatedly.
- Use the same or a separate agent and model for testing and code review, as applicable; prefer a separate agent for normal and complex changes.
- **Never** put secrets, production credentials, or restricted data into an AI tool unless the Company/Client has explicitly approved that environment.
- Human review is **mandatory** for security, authentication/authorization, production data, infrastructure, and other high-impact changes.

Official model references: OpenAI models • Anthropic Claude models • Google Gemini models • GitHub Copilot models

— End of Document —